

INHERITANCES IN JAVA

SINGLE INHERITANCE

CODE

```
class A {
    int x = 10;
    int y = 20;

    void display() {
        System.out.println("this super class");
    }

    void zen() {
        System.out.println("this method is in class a but access through class b object");
    }

    void yvalue() {
        System.out.println(y);
    }
}

class B extends A {
    int x = 9;

    void display() {
        System.out.println("this is super class");
    }

    void sum() {
        System.out.println(this.x + super.x);
    }

    void play() {
        super.display();
    }
}
```

```

public class SingleInheritance {
    public static void main(String[] args) {

        B obj = new B();

        obj.play();
        obj.zen();
        obj.sum();
        obj.display();
        obj.yvalue();
    }
}

```

2. Explanation

- `class A` → **superclass/parent class**.
- `class B extends A` → B inherits the properties and methods of A.
- `int x = 9` in B and `int x = 10` in A show that both classes can have variables with the same name.
- `this.x` → refers to `x` of the **current class B** → 9.
- `super.x` → refers to `x` of the **parent class A** → 10.
- Therefore, `this.x + super.x = 9 + 10 = 19`.
- `super.display()` calls the `display()` method of parent class A.
- `obj.zen()` works because B inherits `zen()` from A.
- `obj.yvalue()` works because B inherits `yvalue()` from A.
- `obj.display()` calls B's `display()` because B **overrides** the method.

3. Output

```

this super class
this method is in class a but access through class b object
19
this is super class
20

```

OUTPUT

```
PS C:\Users\navee\OneDrive\Desktop\programs> javac SingleInheritance.java
PS C:\Users\navee\OneDrive\Desktop\programs> java SingleInheritance
this super class
this method is in class a but access through class b object
19
this is super class
20
```

MULTILEVEL INHERITANCE

CODE

```
class A {
    int x = 10;
    int y = 20;

    void sum() {
        System.out.println("sum is " + (x + y));
    }
}

class B extends A {
    int y = 30;

    void display() {
        System.out.println("this from class b");
    }

    void play() {
        System.out.println("value of x from class a " + x);
    }
}

class C extends B {
    void show() {
        System.out.println("value of y from class b is " + y);
    }
}

public class MultilevelInheritance {
```

```

public static void main(String[] args) {
    C obj = new C();

    obj.show();
    obj.play();
    obj.display();
    obj.sum();
}
}

```

Explanation

- A is the **parent/superclass**.
- B **extends** A → B inherits from A.
- C **extends** B → C inherits from B.
- Therefore, C can access members of both **B and A**.
- **obj.show()** → method of C.
- **obj.play()** → method of B.
- **obj.display()** → method of B.
- **obj.sum()** → method inherited from A.
- In B, **y = 30** hides A's **y = 20** when accessed simply as **y** from B/C.

Output

value of y from class b is 30
 value of x from class a 10
 this from class b
 sum is 30

```

PS C:\Users\navvee\OneDrive\Desktop\programs> java MultilevelInheritance
value of y from class b is30
value of x from class a 10
this from class b
sum is 30

```

Hierarchical Inheritance

```

class A {
    int x = 10;
    int y = 30;

    void sum() {

```

```

        System.out.println("sum is " + (x + y));
    }
}

class B extends A {
    void display() {
        System.err.println("this one sub class");
    }

    void con() {
        System.out.println(x);
    }
}

class C extends A {
    void play() {
        System.out.println("this is second sub class");
    }

    void show() {
        System.out.println(y);
    }
}

public class Hierarchical {
    public static void main(String[] args) {
        C obj = new C();
        B obj1 = new B();

        obj.sum();
        obj1.sum();
        obj.play();
    }
}

```

Explanation

The inheritance structure is:

```

    A
  /\
 B  C

```

- **A** is the **parent class**.

- B and C are **child classes**.
- B extends A → B inherits A.
- C extends A → C inherits A.
- Both B and C can access x, y, and sum() from A.
- obj.sum() → calls sum() inherited by C.
- obj1.sum() → calls sum() inherited by B.
- obj.play() → calls the method belonging to C.

Output

sum is 40

sum is 40

this is second sub class

```
PS C:\Users\navvee\OneDrive\Desktop\programs> javac Hierarchical.java
PS C:\Users\navvee\OneDrive\Desktop\programs> java Hierarchical
sum is 40
sum is 40
this is second sub class
```

MULTI INHERITANCE

```
interface A {
    void showA();
}
```

```
interface B {
    void showB();
}
```

```
class C implements A, B {
    public void showA() {
        System.out.println("this inherit from interface a");
    }

    public void showB() {
        System.out.println("inherit from interface b");
    }

    void showC() {
        System.out.println("this from class c");
    }
}
```

```

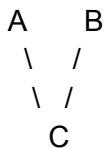
public class MultiInheritance {
    public static void main(String[] args) {
        C obj = new C();

        obj.showA();
        obj.showB();
        obj.showC();
    }
}

```

Explanation

Structure:



- A is an interface containing `showA()`.
- B is another interface containing `showB()`.
- C implements A, B means class C implements **both interfaces**.
- Since interface methods are `public` by default, the implemented methods in C must also be `public`.
- `showC()` is a normal method belonging directly to class C.
- `obj.showA()` calls the method inherited from interface A.
- `obj.showB()` calls the method inherited from interface B.
- `obj.showC()` calls C's own method.

Output

this inherit from interface a

inherit from interface b

this from class c

```

PS C:\Users\navsee\OneDrive\Desktop\programs> javac MultiInheritance.java
PS C:\Users\navsee\OneDrive\Desktop\programs> java MultiInheritance
this inherit from interface a
inherit from interface b
this from class c

```

HYBRID INHERITANCE

```
interface A {
    void showA();
}

interface B extends A {
    void showB();
}

interface C extends A {
    void showC();
}

class D implements B, C {
    public void showA() {
        System.out.println("THIS FROM INTERFACE A");
    }

    public void showB() {
        System.out.println("INHERIT FROM INTERFACE B");
    }

    public void showC() {
        System.out.println("INHERIT FROM INTERFACE C");
    }

    void showD() {
        System.out.println("this from class D");
    }
}

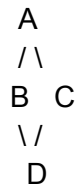
public class HybridInheritance {
    public static void main(String[] args) {
        D obj = new D();

        obj.showA();
        obj.showB();
        obj.showC();
        obj.showD();
    }
}
```

```
}
```

Explanation

The structure is:



- A is the **base interface**.
- B extends A $\rightarrow$ B inherits `showA()` from A.
- C extends A $\rightarrow$ C also inherits `showA()` from A.
- D implements B, C $\rightarrow$ D implements both B and C.
- Since B and C both come from A, D must implement `showA()`, `showB()`, and `showC()`.
- `showD()` is the method directly defined inside class D.
- This combination of **hierarchical + multiple inheritance** is called **hybrid inheritance**.

Output

```
THIS FROM INTERFACE A
INHERIT FROM INTERFACE B
INHERIT FROM INTERFACE C
this from class D
```

```
PS C:\Users\nav...> javac HybridInheritance.java
PS C:\Users\nav...> java HybridInheritance
THIS FROM INTERFACE A
INHERIT FROM INTERFACE B
INHERIT FROM INTERFACE C
this from class D
```